a.k.a. "stop calling the broken thing"

Circuit Breaker

TL;DR

Wrap every call to a remote dependency in a little switch. When too many calls fail, the switch **trips open** and later calls fail *instantly* with a fallback, instead of waiting. After a cool-down it lets a few test calls through. If they work, it **closes** again.

Family: *resilience / stability* | **Lives in:** *the caller (client, gateway or service mesh)*

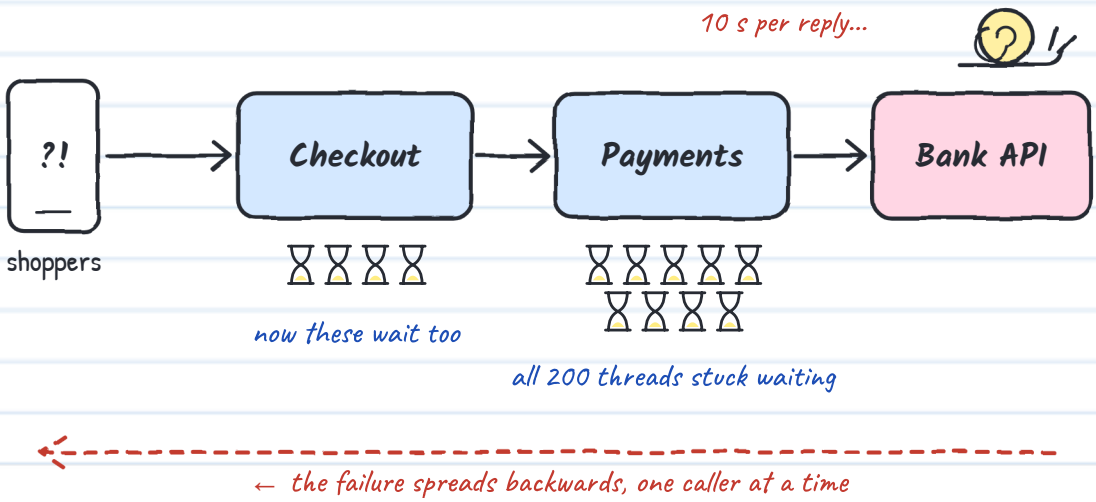
Popularised by: *Michael Nygard, Release It! (2007)* | **Pairs with:** *timeouts, retries, bulkheads*

1 The problem *why do we even need this?*

In a distributed system, services call each other over the network. Sooner or later one of them gets slow or stops answering. The scary part is what that does to everyone who calls it.

Say **Checkout** calls **Payments**, and Payments calls the **Bank API**. The bank has a bad day and every reply takes 10 seconds. Nothing has "crashed", but:

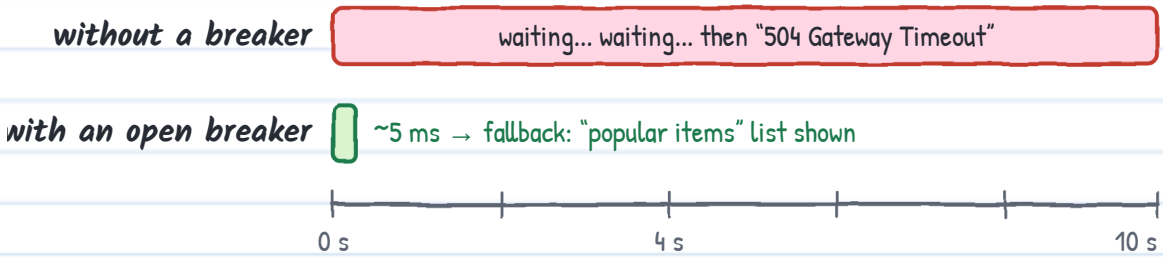
1. Each Payments request now holds a thread (and a DB connection, and memory) for 10 s while it waits.
2. Payments has a pool of 200 threads. New requests queue up behind the stuck ones.
3. Checkout's calls to Payments now also wait, so *Checkout's* threads fill up too.
4. Shoppers see spinners, hit refresh, and send *even more* requests. Retries pile on.
5. The whole site is down because one outside API got slow. This is a **cascading failure**.



▲ *Nothing is "down". One dependency is just slow, and the waiting spreads upstream.*

> ↳ *quick math from class: 50 orders/s × 10 s wait = 500 calls in flight, but only 200 threads → the pool is full after about 4 seconds!*

Key insight: a slow dependency is more dangerous than a dead one. A dead one fails fast ("connection refused"). A slow one quietly holds your resources hostage until the timeout.

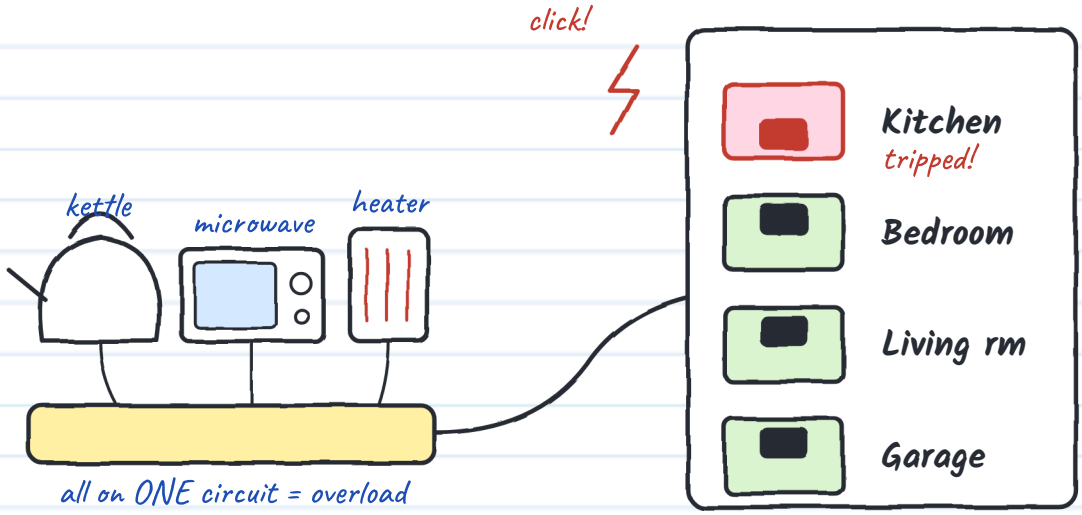


▲ *Failing fast frees the thread right away and the shopper still sees something useful.*

2 Real-life analogy *the breaker panel at home*

The name comes straight from the electrical panel in your house. Plug a kettle, a microwave and a heater into one kitchen circuit and switch them all on. The current spikes, the wire starts to heat up, and **click**: the kitchen breaker trips.

- The kitchen goes dark, but the **house doesn't catch fire** and the bedroom lights stay on. The damage is contained.
- You don't flip it back on every two seconds. You **wait**, unplug the heater, then flip it **once** to test.
- If it trips again right away, something is still wrong, so you leave it off longer.



▲ One circuit trips so the rest of the house keeps its power. Same idea for services.

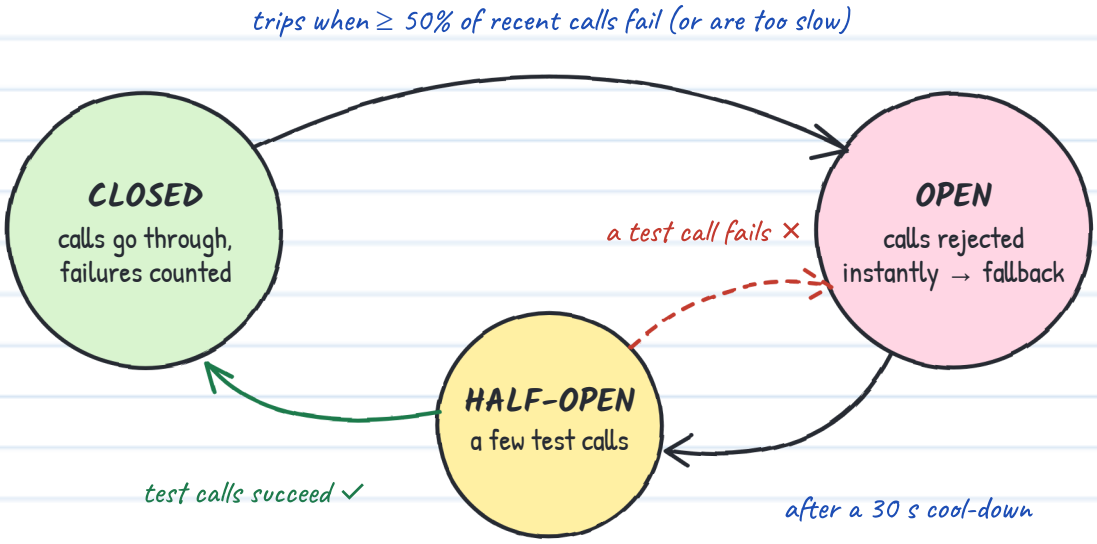
Translating the house to software

At home	In your system
Too much current on one circuit →	Too many failed or slow calls to one dependency
The breaker trips (switch goes down) →	The breaker goes **OPEN** and stops sending calls
Kitchen is dark, the rest of the house is fine →	One feature is degraded, the rest of the app keeps working
Use a camping lamp in the kitchen for now →	**Fallback**: cached data, a default, or a friendly message
Wait a bit, then flip it once to test →	**HALF-OPEN**: after a cool-down, let a few test calls through
It stays on, so you go back to normal →	Test calls succeed, so the breaker goes back to **CLOSED**

> ↳ *another way to remember it: a friend who never picks up. After 5 missed calls you stop calling for an hour, then send ONE text to check. You don't ring 50 times in a row.*

3 How it solves it *three states*

The breaker sits between the caller and the dependency and watches the results of recent calls. It's a small **state machine**:



▲ *Green = normal. Pink = protecting everyone. Yellow = carefully testing the water.*

1 CLOSED (normal)

Every call passes through. The breaker records each result in a **sliding window**, for example the last 20 calls or the last 10 seconds. Timeouts, connection errors and 5xx responses count as failures. Calls slower than a threshold (say 2 s) can count as "slow calls".

2 Trip → OPEN

Once the window has enough calls (a **minimum** like 20, so 2 failures at 3 a.m. don't trip it) and the failure rate or slow-call rate crosses the threshold (say 50%), the breaker opens.

3 OPEN (fail fast)

Calls don't touch the network at all. The breaker throws a "circuit open" error right away, and the caller runs its **fallback**. The dependency gets breathing room to recover instead of being hammered.

4 Cool-down → HALF-OPEN

After a wait (say 30 s) the breaker lets a small number of **trial calls** through (say 3). Everything else is still rejected.

5 Decide

If the trial calls succeed, go back to CLOSED and reset the counters. If any of them fails, go straight back to OPEN and start the cool-down again. Some libraries make each new cool-down longer (exponential back-off).

What can a fallback be?

Cached / stale data

Show yesterday's exchange rate or the last known stock level, with a "may be out of date" note.

A sensible default

Recommendations down? Show the bestsellers list. Avatar service down? Show initials.

Do it later

Put the request on a queue ("we'll email your receipt") and process it when the dependency is back.

Fail clearly

If nothing sensible exists, return a fast, honest error (503 + `Retry-After`) instead of a 30 s hang.

Vocabulary for the exam

Failure-rate threshold

% of failed calls in the window that trips the breaker (often 50%).

Slow-call threshold

A call slower than this counts as "slow". Too many slow calls also trip it.

Sliding window

The recent calls the breaker judges by: count-based (last N) or time-based (last N seconds).

Minimum calls

Don't decide anything until the window holds at least this many calls.

Wait duration (open)

How long to stay OPEN before testing again.

Permitted calls (half-open)

How many trial calls to let through when testing.

circuit-breaker.ts — *simplified, count-based window*

```ts
type State = "CLOSED" | "OPEN" | "HALF_OPEN";

export class CircuitBreaker {
  private state: State = "CLOSED";
  private results: boolean[] = [];    // true = failure
  private openedAt = 0;
  private trials = 0;

  constructor(private opts = {
    windowSize: 20, minCalls: 20, failureRate: 0.5,
    openMs: 30_000, trialCalls: 3,
  }) {}

  async call<T>(fn: () => Promise<T>, fallback: () => T): Promise<T> {
    if (this.state === "OPEN") {
      if (Date.now() - this.openedAt < this.opts.openMs) return fallback(); // fail fast
      this.state = "HALF_OPEN"; this.trials = 0;
    }
    if (this.state === "HALF_OPEN" && this.trials++ >= this.opts.trialCalls) return fallback();
    try {
      const value = await fn();          // fn MUST have its own timeout
      this.record(false);
      return value;
    } catch (err) {
      if (!isRemoteFailure(err)) throw err;  // 400/404 are not the dependency's fault
      this.record(true);
      return fallback();
    }
  }

  private record(failed: boolean) {
    if (this.state === "HALF_OPEN") {
      if (failed) return this.trip();
      if (this.trials >= this.opts.trialCalls) { this.state = "CLOSED"; this.results = []; }
      return;
    }
    this.results = [...this.results, failed].slice(-this.opts.windowSize);
    const fails = this.results.filter(Boolean).length;
    if (this.results.length >= this.opts.minCalls &&
```

```
        fails / this.results.length >= this.opts.failureRate) this.trip();
    }

    private trip() { this.state = "OPEN"; this.openedAt = Date.now(); log("breaker
OPEN"); }
}
```

↳ *in real projects don't hand-roll this. Use Resilience4j (Java), Polly (.NET), opossum (Node), gobreaker (Go), or let the service mesh do it (Envoy / Istio "outlier detection").*

4 Gains vs losses *nothing is free*

What we gain

+ **Stops cascading failures.** One bad dependency can't take the whole system down.
+ **Fails fast.** Answers in milliseconds instead of hanging for a timeout, so threads and connections are freed.
+ **Gives the sick service room to recover** instead of hammering it with traffic and retries.
+ **Graceful degradation.** Users get a fallback instead of an error page.
+ **A clear health signal.** "Breaker opened" is an easy thing to alert on and graph.

What we lose

− **More things to tune.** Thresholds too tight and it flaps open on a blip; too loose and it never trips.
− **Degraded answers.** While OPEN, users get stale or partial data, which must be acceptable for the business.
− **Inconsistent views.** 20 instances = 20 separate breakers, each tripping at a different moment.
− **Harder testing.** You need fault injection to prove each state and the fallback actually work.
− **Can hide problems** if nobody watches it. The fallback looks "fine" while the dependency burns.

5 Architecture ratings *1 = poor · 5 = great*

Same nine characteristics we use for architecture styles. Each line compares a typical service **without** a breaker to the same service **with** one.

Characteristic	With the pattern	Before → after	Why
Simplicity *easy to understand?*		4 → 3 ▼ −1	One more moving part, three states and several settings to understand.

Characteristic	With the pattern	Before → after	Why
Low cost *cheap to build & run?*		4 → 4 = no change	Free libraries, tiny CPU overhead. Mostly costs tuning time.
Deployability *safe, easy releases?*		4 → 4 = no change	No real change. Config can live in a file or the mesh.
Testability *test with confidence?*		4 → 3 ▼ −1	Each state and each fallback needs fault-injection tests.
Scalability *more users & data?*		3 → 3 = no change	Doesn't add capacity by itself. It protects the capacity you have.
Elasticity *sudden spikes?*		2 → 3 ▲ +1	Sheds load from an overloaded dependency, buying autoscaling some time.
Fault tolerance *failure stays contained?*		2 → 5 ▲ +3	The whole point. One slow dependency no longer sinks every caller.
Performance *fast typical request?*		3 → 4 ▲ +1	Normal path unchanged. During incidents, ms instead of a 10 s hang.
Evolvability *easy to change?*		4 → 4 = no change	Wraps a call site. Easy to add, remove or retune later.

 *kept*  *gained*  *lost* *ratings are relative guidance for a typical system, not laws*

6 Common pitfalls *teacher said these show up in incidents a LOT*



No timeout under the breaker

The breaker only counts a failure when a call ends. With a 60 s default timeout, it takes minutes to trip and threads are stuck the whole time. **Fix** → always pair it with a tight timeout on each call.



One breaker for everything

A single breaker around the whole Payments client trips for *all* endpoints when only `/refunds` is broken. **Fix** → one breaker per dependency, and per endpoint when they fail independently.



Counting the wrong errors

A burst of 400s or 404s from bad user input trips the breaker, even though the dependency is healthy. **Fix** → count only timeouts, connection errors, 5xx and 429.



Deciding on tiny samples

At 3 a.m., 2 of 3 calls fail, that's 66%, and the breaker trips for nothing. **Fix** → set a minimum number of calls before the rate counts.

Retrying a "circuit open" error

A retry loop wraps the breaker and keeps retrying the instant rejection, burning CPU and filling logs. **Fix** → treat "breaker open" as non-retryable and go to the fallback.

A fallback that is just as fragile

The fallback calls *another* remote service that is overloaded too, so you've moved the problem. **Fix** → keep fallbacks local and cheap: cache, default, queue.

A silent breaker

It opens and closes all day and nobody notices, because the fallback hides it. **Fix** → log and emit metrics on every state change, and alert when it stays open.

7 When to use it (and when not)

Use it when...

- You make **synchronous** calls (HTTP, gRPC) to services you don't control or that can get slow.
- Calling third-party APIs: payment providers, maps, SMS, shipping rates.
- One page or request fans out to many services.
- A sensible fallback exists, or failing fast is better than waiting.

Skip it when...

- The call is in-process or to a local resource (a file, an in-memory cache).
- Work is async through a queue. Use retries with back-off and a dead-letter queue instead.
- You're using it to avoid fixing a dependency that is always broken.
- Traffic is so low that the window never fills enough to decide anything.

Related patterns

Timeouts & retries · the breaker's best friend

Bulkhead · separate thread pools per dependency

Rate limiter · protects the callee from you

Fallback / graceful degradation

Health checks & outlier detection

8 Self-check quiz

Q Why is a slow dependency worse than a dead one? ✓

A dead service fails fast (connection refused), so callers move on. A slow one holds every caller's threads and connections until the timeout, so the waiting piles up and spreads upstream.

Q The breaker is HALF-OPEN and one trial call fails. What happens?

It goes straight back to OPEN and the cool-down timer restarts. Some libraries also make the next cool-down longer.

Q You have 20 pods, each with its own breaker. Is that a problem?

Usually it's fine. Each pod learns on its own, so they trip at slightly different times and some keep sending traffic for a while. If you need one shared view, do it at the mesh or gateway level (outlier detection) instead of sharing state between pods.

Q Should a 404 count as a failure?

No. A 404 means the request was wrong or the item doesn't exist, and the dependency is healthy. Count timeouts, connection errors, 5xx and 429.

remember!!

Fail fast, fall back, find out, then try again carefully. A breaker protects the caller and gives the dependency room to heal.

Further reading

- Michael T. Nygard, *Release It!*, 2nd ed., chapter "Stability Patterns".
- Martin Fowler, "CircuitBreaker" (martinfowler.com/bliki/CircuitBreaker.html).
- Microsoft Azure Architecture Center, "Circuit Breaker pattern".
- Resilience4j docs, "CircuitBreaker" (sliding windows, slow-call thresholds).